DAA PRACTICAL FILE

DESIGN AND ANALYSIS OF ALGORITHMS (01AI0506)

PRACTICAL – 1: Implementation and Time Analysis of Sorting Algorithms

Objective: *Implement and compare the performance of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort; analyze time complexity and execution time of each to identify the most suitable algorithm for different applications.*

1. Bubble Sort

Algorithm:

1. Start with an unsorted array of n elements.
2. Repeat a pass over the array from index 0 to n-2.
3. In each pass, compare each pair of adjacent elements arr[j] and arr[j+1].
4. If arr[j] > arr[j+1], swap the two elements.
5. If no swaps occur during a full pass, the array is already sorted — stop early.
6. Repeat the passes until the array is completely sorted.

Code:

```
#include <iostream>
using namespace std;
void bubbleSort(int arr[], int n) {
    bool swapped;
    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
void printArray(int arr[], int n) { for (int i = 0; i < n; i++) cout << arr[i] << " "; cout << endl; }
int main() {
    int n; cout << "Enter the number of elements: "; cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    bubbleSort(arr, n);
    cout << "Sorted array: "; printArray(arr, n);
    cout << "Roll No: 92460118370" << endl;
    delete[] arr; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter the number of elements: 5
Enter 5 elements: 8 6 5 7 4
Sorted array: 4 5 6 7 8
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
Best Case	O(n)	Array already sorted; with the swapped flag, only one pass is required.

Case	Time Complexity	Description
Average Case	$O(n^2)$	Random order requires multiple comparisons and swaps.
Worst Case	$O(n^2)$	Reverse sorted array requires the maximum comparisons and swaps.

Space Complexity: O(1)

2. Selection Sort

Algorithm:

1. For i = 0 to n-2, assume arr[i] is the minimum of the unsorted part.
2. Scan the remaining unsorted elements (j = i+1 to n-1) to find the actual minimum.
3. Swap the minimum element found with arr[i].
4. Move the boundary of the sorted part one step forward.
5. Repeat until the entire array is sorted.

Code:

```
#include <iostream>
using namespace std;
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) if (arr[j] < arr[minIdx]) minIdx = j;
        int temp = arr[i]; arr[i] = arr[minIdx]; arr[minIdx] = temp;
    }
}
void printArray(int arr[], int n) { for (int i = 0; i < n; i++) cout << arr[i] << " "; cout << endl; }
int main() {
    int n; cout << "Enter the number of elements: "; cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    selectionSort(arr, n);
    cout << "Sorted array: "; printArray(arr, n);
    cout << "Roll No: 92460118370" << endl;
    delete[] arr; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter the number of elements: 5
Enter 5 elements: 64 25 12 22 11
Sorted array: 11 12 22 25 64
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
Best Case	$O(n^2)$	Requires n(n-1)/2 comparisons regardless of initial ordering.
Average Case	$O(n^2)$	Scans unsorted subarray for minimum element in each pass.
Worst Case	$O(n^2)$	Identical comparison count even for reverse sorted input.

Space Complexity: O(1)

3. Insertion Sort

Algorithm:

1. Start from the second element (index 1) and treat it as the key.
2. Compare the key with elements before it in the sorted part.
3. Shift all elements greater than the key one position to the right.
4. Insert the key into its correct position.
5. Repeat for every element until the whole array is sorted.

Code:

```
#include <iostream>
using namespace std;
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i]; int j = i - 1;
        while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; j--; }
        arr[j + 1] = key;
    }
}
void printArray(int arr[], int n) { for (int i = 0; i < n; i++) cout << arr[i] << " "; cout << endl; }
int main() {
    int n; cout << "Enter the number of elements: "; cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    insertionSort(arr, n);
    cout << "Sorted array: "; printArray(arr, n);
    cout << "Roll No: 92460118370" << endl;
    delete[] arr; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter the number of elements: 5
Enter 5 elements: 12 11 13 5 6
Sorted array: 5 6 11 12 13
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
Best Case	O(n)	Array already sorted; inner loop terminates in 1 comparison per element.
Average Case	$O(n^2)$	Requires half array shifts on average per element insertion.
Worst Case	$O(n^2)$	Reverse sorted array requires n(n-1)/2 element shifts.

Space Complexity: O(1)

4. Merge Sort

Algorithm:

1. Divide the array into two halves recursively until each sub-array has one element.
2. Merge two sorted halves by repeatedly picking the smaller of the two front elements.
3. Copy any remaining elements from either half once the other is exhausted.
4. Continue merging pairs of sub-arrays until the whole array is merged and sorted.

Code:

```
#include <iostream>
using namespace std;
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int* L = new int[n1]; int* R = new int[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
    delete[] L; delete[] R;
}
```

```
void mergeSort(int arr[], int l, int r) {
    if (l < r) { int m = l + (r - l) / 2; mergeSort(arr, l, m); mergeSort(arr, m + 1, r); merge(arr, l, m, r); }
}
int main() {
    int n; cout << "Enter the number of elements: "; cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    mergeSort(arr, 0, n - 1);
    cout << "Sorted array: "; for (int i = 0; i < n; i++) cout << arr[i] << " "; cout << endl;
    cout << "Roll No: 92460118370" << endl;
    delete[] arr; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter the number of elements: 5
Enter 5 elements: 38 27 43 3 9
Sorted array: 3 9 27 38 43
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
Best Case	O(n log n)	Divides array into halves and merges in linear O(n) time.
Average Case	O(n log n)	Consistent logarithmic split depth log2(n) across all inputs.
Worst Case	O(n log n)	Guaranteed O(n log n) bound solved via Master Theorem.

Space Complexity: O(n)

5. Quick Sort

Algorithm:

1. Choose a pivot element (the last element of the sub-array).
2. Partition the array so elements smaller than the pivot move to its left and larger ones to its right.
3. Place the pivot at its correct sorted position.
4. Recursively apply the same process to the left and right sub-arrays.
5. Combine results — the array becomes sorted in place.

Code:

```
#include <iostream>
using namespace std;
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; int i = low - 1;
    for (int j = low; j < high; j++) if (arr[j] < pivot) { i++; int t = arr[i]; arr[i] = arr[j]; arr[j] = t; }
    int t = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = t; return i + 1;
}
void quickSort(int arr[], int low, int high) {
    if (low < high) { int pi = partition(arr, low, high); quickSort(arr, low, pi - 1); quickSort(arr, pi + 1, high); }
}
int main() {
    int n; cout << "Enter the number of elements: "; cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    quickSort(arr, 0, n - 1);
    cout << "Sorted array: "; for (int i = 0; i < n; i++) cout << arr[i] << " "; cout << endl;
    cout << "Roll No: 92460118370" << endl;
    delete[] arr; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter the number of elements: 5
Enter 5 elements: 10 7 8 9 1
Sorted array: 1 7 8 9 10
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
Best Case	O(n log n)	Partition splits array evenly into sub-problems of size n/2.
Average Case	O(n log n)	Balanced partitioning on random inputs.
Worst Case	$O(n^2)$	Pivot selection consistently yields unbalanced partitions.

Space Complexity: O(log n)

PRACTICAL – 2: Implementation and Time Analysis of Searching Algorithms

Objective: *Implement and compare Linear Search and Binary Search; analyze time complexity and execution steps on unsorted and sorted datasets.*

1. Linear Search

Algorithm:

1. Start scanning from the first element of the array.
2. Compare each element with the target value one by one.
3. If a match is found, return its index immediately.
4. If the end of the array is reached with no match, return -1.

Code:

```
#include <iostream>
using namespace std;
int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) if (arr[i] == target) return i;
    return -1;
}
int main() {
    int n, target; cout << "Enter number of elements: "; cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    cout << "Enter target element to search: "; cin >> target;
    int result = linearSearch(arr, n, target);
    if (result != -1) cout << "Element found at index: " << result << endl;
    else cout << "Element not found" << endl;
    cout << "Roll No: 92460118370" << endl;
    delete[] arr; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50
Enter target element to search: 30
Element found at index: 2
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
Best Case	O(1)	Target element is present at index 0.
Average Case	O(n)	Target element is located at index n/2 on average.
Worst Case	O(n)	Target is at index n-1 or completely absent.

Space Complexity: O(1)

2. Binary Search

Algorithm:

1. Precondition: the array must be sorted.
2. Set low = 0 and high = n-1.
3. Compute mid = low + (high-low)/2 and compare arr[mid] with the target.
4. If arr[mid] equals target, return mid.
5. If target is smaller, search the left half (high = mid-1); otherwise search the right half (low = mid+1).
6. Repeat until the element is found or low > high.

Code:

```
#include <iostream>
using namespace std;
int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1; else high = mid - 1;
    }
    return -1;
}
int main() {
    int n, target; cout << "Enter number of elements (sorted): "; cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " sorted elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    cout << "Enter target element to search: "; cin >> target;
    int result = binarySearch(arr, n, target);
    if (result != -1) cout << "Target " << target << " found at index: " << result << endl;
    else cout << "Target element not found" << endl;
    cout << "Roll No: 92460118370" << endl;
    delete[] arr; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of elements (sorted): 10
Enter 10 sorted elements: 12 25 34 45 50 64 78 88 90 99
Enter target element to search: 78
Target 78 found at index: 6
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
Best Case	O(1)	Target element is located at the exact middle index mid.
Average Case	O(log n)	Search space is halved at each iteration.
Worst Case	O(log n)	Target is at extreme boundary or absent.

Space Complexity: O(1)

PRACTICAL – 3: Implementation of Max Heap Sort

Objective: *Implement Max Heap Sort using binary heap operations (heapify and build-max-heap) and analyze its time and space complexity.*

1. Max Heap Sort

Algorithm:

1. Build a max heap from the input array by heapifying from the last non-leaf node up to the root.
2. Swap the root (the maximum element) with the last element of the heap.
3. Reduce the heap size by 1 and heapify the root again to restore the max-heap property.
4. Repeat the swap-and-heapify steps until the heap size becomes 1.
5. The array is now sorted in ascending order.

Code:

```
#include <iostream>
using namespace std;
void heapify(int arr[], int n, int i) {
    int largest = i, left = 2 * i + 1, right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) { int t = arr[i]; arr[i] = arr[largest]; arr[largest] = t; heapify(arr, n, largest); }
}
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) { int t = arr[0]; arr[0] = arr[i]; arr[i] = t; heapify(arr, i, 0); }
}
int main() {
    int n; cout << "Enter number of elements: "; cin >> n;
    int* arr = new int[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    heapSort(arr, n);
    cout << "Sorted Array (Max Heap Sort): "; for (int i = 0; i < n; i++) cout << arr[i] << " "; cout << endl;
    cout << "Roll No: 92460118370" << endl;
    delete[] arr; return 0;
}
```

Output:

```
DEBUG CONSOLE   OUTPUT   TERMINAL   GITLENS   COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of elements: 8
Enter 8 elements: 12 11 13 5 6 7 9 20
Sorted Array (Max Heap Sort): 5 6 7 9 11 12 13 20
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
Best Case	O(n log n)	Building heap takes O(n), extractions take O(n log n).
Average Case	O(n log n)	Guaranteed logarithmic tree height traversal.
Worst Case	O(n log n)	Strict upper bound guaranteed across all inputs.

Space Complexity: O(1)

PRACTICAL – 4: Time Analysis of Factorial (Iterative vs Recursive)

Objective: *Implement Iterative Factorial and Recursive Factorial in C++ and compare performance, execution steps, and memory overhead.*

1. Factorial Analysis

Algorithm:

1. Iterative: initialize result = 1, then multiply result by every integer from 1 to n in a loop; return result.
2. Recursive: base case — if n <= 1, return 1.
3. Recursive case: return n multiplied by factorial(n-1), unwinding the recursion back up to n.

Code:

```
#include <iostream>
using namespace std;
unsigned long long factorialIterative(int n) {
    unsigned long long res = 1;
    for (int i = 1; i <= n; i++) res *= i;
    return res;
}
unsigned long long factorialRecursive(int n) {
    if (n <= 1) return 1;
    return n * factorialRecursive(n - 1);
}
int main() {
    int n; cout << "Enter a non-negative integer: "; cin >> n;
    cout << "Iterative Factorial(" << n << ") = " << factorialIterative(n) << endl;
    cout << "Recursive Factorial(" << n << ") = " << factorialRecursive(n) << endl;
    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter a non-negative integer: 10
Iterative Factorial(10) = 3628800
Recursive Factorial(10) = 3628800
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
Iterative	O(n)	Executes single linear loop with zero call stack overhead.
Recursive	O(n)	Allocates n stack frames on the call stack.

Space Complexity: Iterative: O(1), Recursive: O(n)

PRACTICAL – 5: Implementation of 0/1 Knapsack using Dynamic Programming

Objective: *Solve the 0/1 Knapsack Problem using Dynamic Programming bottom-up tabulation to find maximum profit under capacity constraint W.*

1. 0/1 Knapsack DP

Algorithm:

1. Create a DP table dp[n+1][W+1] initialized to 0.
2. For each item i and each capacity w, decide whether to include or exclude item i.
3. If the item's weight fits within w: dp[i][w] = max(value + dp[i-1][w-weight], dp[i-1][w]).
4. Otherwise: dp[i][w] = dp[i-1][w] (item excluded).
5. The answer, the maximum achievable value, is stored in dp[n][W].

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int knapsackDP(int W, int wt[], int val[], int n) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int w = 1; w <= W; w++)
            dp[i][w] = (wt[i - 1] <= w) ? max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]) : dp[i - 1][w];
    return dp[n][W];
}
int main() {
    int n, W; cout << "Enter number of items: "; cin >> n;
    int* val = new int[n]; int* wt = new int[n];
    cout << "Enter values of " << n << " items: "; for (int i = 0; i < n; i++) cin >> val[i];
    cout << "Enter weights of " << n << " items: "; for (int i = 0; i < n; i++) cin >> wt[i];
    cout << "Enter max capacity W: "; cin >> W;
    cout << "Maximum Profit Value in Knapsack = " << knapsackDP(W, wt, val, n) << endl;
    cout << "Roll No: 92460118370" << endl;
    delete[] val; delete[] wt; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of items: 3
Enter values of 3 items: 60 100 120
Enter weights of 3 items: 10 20 30
Enter max capacity W: 50
Maximum Profit Value in Knapsack = 220
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
All Cases	O(n·W)	Fills DP table of size (n+1) x (W+1) using optimal subproblems.

Space Complexity: O(n·W)

PRACTICAL – 6: Implementation of Matrix Chain Multiplication

Objective: *Implement Matrix Chain Multiplication using Dynamic Programming to determine the optimal parenthesization minimizing scalar multiplications.*

1. Matrix Chain Multiplication

Algorithm:

1. Represent the matrix dimensions as an array p[0..n].
2. For each chain length L from 2 to n, compute the minimum multiplication cost m[i][j] for multiplying matrices i through j.
3. Try every possible split point k between i and j and pick the one that minimizes the cost.
4. Store results in a DP table; m[1][n-1] gives the minimum number of scalar multiplications.

Code:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int matrixChainOrder(int p[], int n) {
    vector<vector<int>> m(n, vector<int>(n, 0));
    for (int L = 2; L < n; L++)
        for (int i = 1; i < n - L + 1; i++) {
            int j = i + L - 1; m[i][j] = INT_MAX;
            for (int k = i; k <= j - 1; k++) {
                int q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (q < m[i][j]) m[i][j] = q;
            }
        }
    return m[1][n - 1];
}
int main() {
    int numMatrices; cout << "Enter number of matrices: "; cin >> numMatrices;
    int* p = new int[numMatrices + 1];
    cout << "Enter dimension array p of size " << numMatrices + 1 << ": ";
    for (int i = 0; i <= numMatrices; i++) cin >> p[i];
    cout << "Minimum Scalar Multiplications Required: " << matrixChainOrder(p, numMatrices + 1) << endl;
    cout << "Roll No: 92460118370" << endl;
    delete[] p; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of matrices: 4
Enter dimension array p of size 5: 10 30 5 60 8
Minimum Scalar Multiplications Required: 4500
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
All Cases	$O(n^3)$	Evaluates chain lengths L from 2 to n with split choices k.

Space Complexity: $O(n^2)$

PRACTICAL – 7: Implementation of Coin Change Problem

Objective: *Solve the Coin Change Problem (minimum coins required to make amount V) using Dynamic Programming bottom-up tabulation.*

1. Coin Change DP

Algorithm:

1. Create a DP array dp[0..amount], initialized to infinity except dp[0] = 0.
2. For each amount i from 1 to the target, try every coin denomination.
3. If a coin's value fits within i, update dp[i] = min(dp[i], dp[i-coin] + 1).
4. The answer is dp[amount]; if it remains infinity, the amount cannot be formed.

Code:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int coinChangeMin(int coins[], int n, int amount) {
    vector<int> dp(amount + 1, INT_MAX); dp[0] = 0;
    for (int i = 1; i <= amount; i++)
        for (int j = 0; j < n; j++)
            if (i >= coins[j] && dp[i - coins[j]] != INT_MAX) dp[i] = min(dp[i], dp[i - coins[j]] + 1);
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
int main() {
    int n, amount; cout << "Enter number of coin denominations: "; cin >> n;
    int* coins = new int[n];
    cout << "Enter " << n << " coin denominations: "; for (int i = 0; i < n; i++) cin >> coins[i];
    cout << "Enter target amount: "; cin >> amount;
    int ans = coinChangeMin(coins, n, amount);
    if (ans != -1) cout << "Minimum coins needed to make amount " << amount << " = " << ans << endl;
    else cout << "Amount cannot be formed" << endl;
    cout << "Roll No: 92460118370" << endl;
    delete[] coins; return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of coin denominations: 3
Enter 3 coin denominations: 1 2 5
Enter target amount: 11
Minimum coins needed to make amount 11 = 3
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
All Cases	O(n·V)	Fills 1D array up to target amount V across n coin types.

Space Complexity: O(V)

PRACTICAL – 8: Implementation of Graph Traversals (DFS & BFS)

Objective: *Implement Depth-First Search (DFS) and Breadth-First Search (BFS) graph traversal algorithms using adjacency lists.*

1. DFS & BFS Graph Traversals

Algorithm:

1. DFS: start at a node, mark it visited, and print it; recursively visit each unvisited neighbor; backtrack when no unvisited neighbors remain.
2. BFS: start at a node, mark it visited, and enqueue it.
3. BFS: dequeue a node, print it, then enqueue all of its unvisited neighbors, marking each visited.
4. BFS: repeat until the queue is empty.

Code:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
void DFS(int node, const vector<vector<int>>& adj, vector<bool>& visited) {
    visited[node] = true; cout << node << " ";
    for (int nb : adj[node]) if (!visited[nb]) DFS(nb, adj, visited);
}
void BFS(int startNode, const vector<vector<int>>& adj, int V) {
    vector<bool> visited(V, false); queue<int> q;
    visited[startNode] = true; q.push(startNode);
    while (!q.empty()) {
        int curr = q.front(); q.pop(); cout << curr << " ";
        for (int nb : adj[curr]) if (!visited[nb]) { visited[nb] = true; q.push(nb); }
    }
}
int main() {
    int V, E; cout << "Enter number of vertices and edges: "; cin >> V >> E;
    vector<vector<int>> adj(V);
    cout << "Enter " << E << " edges (u v):" << endl;
    for (int i = 0; i < E; i++) { int u, v; cin >> u >> v; adj[u].push_back(v); adj[v].push_back(u); }
    int startNode = 0;
    cout << "DFS Traversal starting from node 0: ";
    vector<bool> visited(V, false); DFS(startNode, adj, visited); cout << endl;
    cout << "BFS Traversal starting from node 0: "; BFS(startNode, adj, V); cout << endl;
    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of vertices and edges: 5 6
Enter 6 edges (u v):
0 1
0 2
1 3
1 4
2 4
3 4
DFS Traversal starting from node 0: 0 1 3 4 2
BFS Traversal starting from node 0: 0 1 2 3 4
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
DFS	O(V + E)	Visits every node and edge once in the adjacency list.
BFS	O(V + E)	Enqueues every node and edge once, level-by-level.

Space Complexity: O(V)

PRACTICAL – 9: Implementation of Prim's Algorithm for Minimum Spanning Tree

Objective: *Implement Prim's Greedy Algorithm using priority queues to compute the Minimum Spanning Tree (MST) of a weighted connected graph.*

1. Prim's Algorithm

Algorithm:

1. Initialize the key value of every vertex to infinity, except the start vertex, whose key is 0.
2. Use a min-priority queue to repeatedly extract the vertex with the smallest key that is not yet in the MST.
3. Add the extracted vertex to the MST and update the key values of its adjacent vertices if a smaller edge weight is found.
4. Repeat until all vertices are included in the MST; sum of chosen keys gives the total MST weight.

Code:

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
using namespace std;
typedef pair<int, int> pii;
void primsMST(int V, const vector<vector<pii>>& adj) {
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    vector<int> key(V, INT_MAX); vector<int> parent(V, -1); vector<bool> inMST(V, false);
    pq.push({0, 0}); key[0] = 0; int totalWeight = 0;
    while (!pq.empty()) {
        int u = pq.top().second; pq.pop();
        if (inMST[u]) continue;
        inMST[u] = true; totalWeight += key[u];
        for (auto& edge : adj[u]) {
            int v = edge.second, weight = edge.first;
            if (!inMST[v] && weight < key[v]) { key[v] = weight; pq.push({key[v], v}); parent[v] = u; }
        }
    }
    cout << "Total Minimum Spanning Tree Weight = " << totalWeight << endl;
}
int main() {
    int V, E; cout << "Enter number of vertices and edges: "; cin >> V >> E;
    vector<vector<pii>> adj(V);
    cout << "Enter " << E << " edges (u v weight):" << endl;
    for (int i = 0; i < E; i++) { int u, v, w; cin >> u >> v >> w; adj[u].push_back({w, v}); adj[v].push_back({w, u});
}
    primsMST(V, adj);
    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of vertices and edges: 5 7
Enter 7 edges (u v weight):
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9
Total Minimum Spanning Tree Weight = 16
Roll No: 92460118370
```

```
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
All Cases	O((V+E) log V)	Priority queue heap updates per edge.

Space Complexity: O(V + E)

PRACTICAL – 10: Implementation of Kruskal's Algorithm for MST

Objective: *Implement Kruskal's Algorithm for Minimum Spanning Tree using Disjoint Set Union (DSU) with path compression.*

1. Kruskal's Algorithm

Algorithm:

1. Sort all edges of the graph in non-decreasing order of weight.
2. Initialize a Disjoint Set Union (DSU) with every vertex in its own set.
3. For each edge (u, v) in sorted order, if u and v belong to different sets, add the edge to the MST and union the two sets.
4. Skip the edge if u and v are already in the same set (it would form a cycle).
5. Repeat until exactly (V-1) edges have been added to the MST.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
struct Edge { int u, v, weight; bool operator<(const Edge& o) const { return weight < o.weight; } };
class DSU {
    vector<int> parent;
public:
    DSU(int n) : parent(n) { for (int i = 0; i < n; i++) parent[i] = i; }
    int find(int i) { if (parent[i] == i) return i; return parent[i] = find(parent[i]); }
    bool unite(int i, int j) {
        int ri = find(i), rj = find(j);
        if (ri != rj) { parent[rj] = ri; return true; }
        return false;
    }
};
void kruskalsMST(int V, vector<Edge>& edges) {
    sort(edges.begin(), edges.end());
    DSU dsu(V); int totalWeight = 0;
    for (const auto& e : edges) if (dsu.unite(e.u, e.v)) totalWeight += e.weight;
    cout << "Total Minimum Spanning Tree Weight = " << totalWeight << endl;
}
int main() {
    int V, E; cout << "Enter number of vertices and edges: "; cin >> V >> E;
    vector<Edge> edges(E);
    cout << "Enter " << E << " edges (u v weight):" << endl;
    for (int i = 0; i < E; i++) cin >> edges[i].u >> edges[i].v >> edges[i].weight;
    kruskalsMST(V, edges);
    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of vertices and edges: 4 5
Enter 5 edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Total Minimum Spanning Tree Weight = 19
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
All Cases	O(E log E)	Edge sorting dominates the overall complexity.

Space Complexity: O(V + E)

PRACTICAL – 11: Implementation of Floyd-Warshall Algorithm

Objective: *Implement the Floyd-Warshall Algorithm for computing All-Pairs Shortest Paths (APSP) using Dynamic Programming matrix operations.*

1. Floyd-Warshall Algorithm

Algorithm:

1. Initialize the distance matrix with the given edge weights, using INF where no direct edge exists.
2. For each vertex k (used as an intermediate), update dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]) for every pair (i, j).
3. Repeat this update for all k from 0 to V-1.
4. The resulting matrix holds the shortest distance between every pair of vertices.

Code:

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;
const int INF = 99999;
void floydWarshall(int V, vector<vector<int>>& dist) {
    for (int k = 0; k < V; k++)
        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                if (dist[i][k] < INF && dist[k][j] < INF && dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
}
int main() {
    int V; cout << "Enter number of vertices: "; cin >> V;
    vector<vector<int>> dist(V, vector<int>(V));
    cout << "Enter adjacency matrix (" << V << "x" << V << ", use 99999 for INF):" << endl;
    for (int i = 0; i < V; i++) for (int j = 0; j < V; j++) cin >> dist[i][j];
    floydWarshall(V, dist);
    cout << "ALL-PAIRS SHORTEST PATH MATRIX:" << endl;
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) { if (dist[i][j] >= INF) cout << setw(8) << "INF"; else cout << setw(8) << dist[i]
[j]; }
        cout << endl;
    }
    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of vertices: 4
Enter adjacency matrix (4x4, use 99999 for INF):
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0
ALL-PAIRS SHORTEST PATH MATRIX:
       0       5       8       9
     INF       0       3       4
     INF     INF       0       1
     INF     INF     INF       0
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
All Cases	$O(V^3)$	Three nested loops over V vertices.

Space Complexity: $O(V^2)$

PRACTICAL – 12: Implementation of Travelling Salesman Problem

Objective: *Solve the Travelling Salesman Problem (TSP) using Held-Karp Bitmask Dynamic Programming to find the minimum tour cost visiting all cities once.*

1. Travelling Salesman Problem (Held-Karp DP)

Algorithm:

1. Represent the set of visited cities using a bitmask, along with the current city position.
2. Base case: if all cities have been visited, return the cost of returning to the start city.
3. Recursive case: try visiting every unvisited city next and take the minimum total cost path.
4. Memoize results in dp[mask][pos] to avoid recomputing the same sub-problem.
5. The final answer is the minimum cost path that starts and ends at city 0.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
const int INF = 1e9;
int tsp(int mask, int pos, int N, const vector<vector<int>>& dist, vector<vector<int>>& dp) {
    if (mask == (1 << N) - 1) return dist[pos][0];
    if (dp[mask][pos] != -1) return dp[mask][pos];
    int ans = INF;
    for (int city = 0; city < N; city++)
        if ((mask & (1 << city)) == 0)
            ans = min(ans, dist[pos][city] + tsp(mask | (1 << city), city, N, dist, dp));
    return dp[mask][pos] = ans;
}
int main() {
    int N; cout << "Enter number of cities: "; cin >> N;
    vector<vector<int>> dist(N, vector<int>(N));
    cout << "Enter " << N << "x" << N << " distance matrix:" << endl;
    for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) cin >> dist[i][j];
    vector<vector<int>> dp(1 << N, vector<int>(N, -1));
    int minCost = tsp(1, 0, N, dist, dp);
    cout << "Minimum Cost to Visit All Cities and Return to Start = " << minCost << endl;
    cout << "Roll No: 92460118370" << endl;
    return 0;
}
```

Output:

```
DEBUG CONSOLE  OUTPUT  TERMINAL  GITLENS  COMMENTS
⌄ TERMINAL
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ % ./a.out
Enter number of cities: 4
Enter 4x4 distance matrix:
0 20 42 25
20 0 30 34
42 30 0 12
25 34 12 0
Minimum Cost to Visit All Cities and Return to Start = 80
Roll No: 92460118370
(base) mandhatisaiganesh@mandhatis-MacBook-Air ~ %
```

Time Complexity:

Case	Time Complexity	Description
All Cases	$O(n^2 \cdot 2^n)$	State space of 2^n masks x n cities.

Space Complexity: $O(n \cdot 2^n)$